
Rapport IA02

Daria EZHOVA - Nina COIFFIN - Pierre RIBET

May 2026

Contents

Partie I	2
Présentation des données	2
Contexte et origine	2
Contenu du dataset	2
Motivation de l'utilisation	2
Entraînement des modèles	3
Régression logistique	3
Random Forest	3
Motivation	3
Fonctionnement	3
Comparaison des résultats	4
CNN's	5
Comparaison des résultats	8
Difficultés rencontrées	8
Partie II	9
Présentation des données	9
Chargement des données	9
Visualisation des données	10
Nettoyage des données	11
Pipeline d'entraînement	11
Modèles	12
CNN	12
CNN BiLSTM	13
CNN 2D & 1D	13
Comparaisons des résultats	14
Augmentation des données	15
Résultats / Impact	15
Nouvelles données	15
Nos échantillons	15
Conclusion	16
Bibliographie	17

Lien vers le repository de ce projet : [GitHub](#)

Partie I

Présentation des données

Le dataset CIFAR-10 est utilisé dans la recherche en vision par ordinateur, pour les tâches de classification d'images. Il permet d'évaluer la performance de différents modèles d'apprentissage automatique, notamment les réseaux de neurones convolutifs (CNN).

Contexte et origine

Le dataset a été développé en 2009, par *Alex Krizhevsky, Vinod Nair et Geoffrey Hinton* dans le cadre de travaux sur l'apprentissage profond et la reconnaissance d'images. Il est disponible sur le site officiel CIFAR Dataset il a été utilisé dans des concours et benchmarks internationaux en vision par ordinateur.

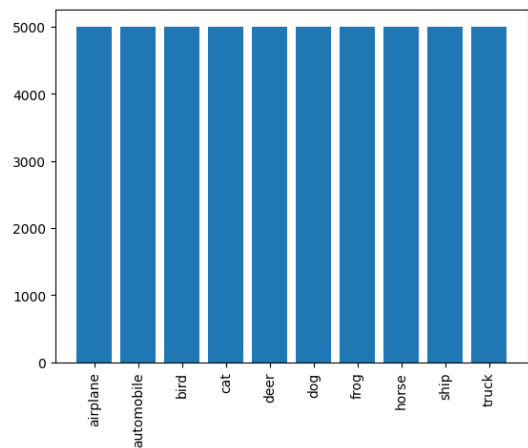
Son objectif principal est de fournir un jeu de données suffisamment complexe pour tester des modèles de classification tout en restant accessible pour l'entraînement rapide, grâce à la taille modérée des images.

Contenu du dataset

Le CIFAR-10 contient 60 000 images en couleur, de dimension 32x32 pixels, réparties en 10 classes distinctes représentant différents objets et animaux.

Classe (anglais)	Classe (français)
Airplane	Avion
Automobile	Voiture
Bird	Oiseau
Cat	Chat
Deer	Cerf
Dog	Chien
Frog	Grenouille
Horse	Cheval
Ship	Bateau
Truck	Camion

Correspondance des classes du dataset CIFAR-10 en anglais et en français.



Répartition équilibrée des classes dans le dataset CIFAR-10.

Une vérification rapide de la répartition des classes montre qu'aucune catégorie n'est déséquilibrée en nombre d'échantillons (cf. Figure).

Les concepteurs du dataset ont pris soin à ce que Voiture et Camion, des classes proches visuellement dans notre imaginaire, sont considérées comme strictement distinctes (*Les voitures n'incluent pas de véhicules pouvant être confondu avec des camions*), ce qui évite la confusion pour les modèles de classification.

Motivation de l'utilisation

L'étude de ce dataset permet de :

- Tester et comparer des modèles de classification standard et avancés.
- Explorer l'impact de la simplification des données (conversion en niveaux de gris) sur les performances.
- Développer des pipelines d'apprentissage automatique robustes, applicables ensuite à des images plus grandes ou à des applications réelles.

Le CIFAR-10 est donc un excellent terrain de test pour la classification d'images, d'autant plus qu'il n'est qu'un sous-ensemble d'un dataset labélisé de 80 millions d'images (Tiny Images). Il permet ainsi de faire un premier pas dans le domaine de la vision par ordinateur.

Entraînement des modèles

Régression logistique

La régression logistique est un algorithme de classification supervisée largement utilisé pour les problèmes de classification binaire et multiclasse. Malgré son nom, il ne s'agit pas d'un algorithme de régression mais bien d'un classifieur. Son principe consiste à associer à chaque classe une probabilité d'appartenance en fonction des caractéristiques d'entrée. Dans le cas du dataset CIFAR-10, chaque image est transformée en un vecteur contenant l'ensemble des pixels de l'image. La décision finale est obtenue en sélectionnant la classe possédant la probabilité la plus élevée. La régression logistique est un modèle linéaire : elle cherche à établir une frontière de décision entre les différentes classes à partir d'une combinaison linéaire des caractéristiques.

Random Forest

La forêt aléatoire est un algorithme d'ensemble basé sur la combinaison de nombreux arbres de décision. Chaque arbre est entraîné sur une partie différente des données et réalise sa propre prédiction. La décision finale est obtenue grâce à un vote majoritaire entre tous les arbres. Contrairement à la régression logistique, Random Forest est capable de modéliser des relations non linéaires et de capturer des interactions complexes entre les variables. Cette caractéristique le rend souvent plus performant sur des problèmes complexes, mais également plus susceptible de surapprendre les données d'entraînement.

Motivation

Le choix de ces deux algorithmes repose sur plusieurs critères. Tout d'abord, ils appartiennent à deux familles de modèles différents. La régression logistique représente une approche linéaire simple tandis que Random Forest représente une approche non linéaire basée sur des arbres de décision. Cette différence permet de comparer deux philosophies d'apprentissage distinctes. Ensuite, ces modèles sont relativement rapides à entraîner sur un ordinateur personnel. Le dataset CIFAR-10 contient 50 000 images d'entraînement et 10 000 images de test. Certains algorithmes classiques comme les SVM à noyau ou les méthodes basées sur les plus proches voisins peuvent devenir particulièrement coûteux en temps de calcul sur un dataset de cette taille. La régression logistique et Random Forest constituent donc un bon compromis entre simplicité, temps d'exécution et performances. Par ailleurs, ces deux algorithmes servent de référence pour évaluer les difficultés rencontrées lorsqu'on applique des méthodes de machine learning classiques à un problème de classification d'images. Les images sont naturellement organisées sous forme de matrices de pixels possédant une structure spatiale. Or, ces algorithmes ne sont pas conçus pour exploiter cette structure. Chaque image est simplement transformée en un vecteur de valeurs numériques, ce qui entraîne une perte d'information sur les relations entre les pixels voisins.

Fonctionnement

Les deux programmes suivent globalement la même démarche. La première étape consiste à charger le dataset CIFAR-10. Les algorithmes de machine learning classiques ne pouvant pas traiter directement le format des images de ce dataset, chaque image est transformée en un vecteur de 3072 valeurs ($32 \times 32 \times 3 = 3072$). Cette opération est appelée "flattening". Une réduction de dimension est ensuite réalisée à l'aide de l'algorithme PCA (Principal Component Analysis). Cette méthode permet de conserver l'essentiel de l'information tout en réduisant fortement le nombre de caractéristiques à traiter. Cette étape réduit le temps d'entraînement et limite les besoins en mémoire. Dans le cas de la régression logistique, une phase de normalisation est également appliquée afin de rendre les différentes variables comparables. Les données transformées sont ensuite utilisées pour entraîner le modèle. Une fois l'entraînement terminé, les modèles effectuent des prédictions sur les données d'entraînement et sur les données de test. Plusieurs indicateurs sont alors calculés :

- La précision sur l'ensemble d'entraînement (Train Accuracy),
- La précision sur l'ensemble de test (Test Accuracy),
- La fonction de perte sur l'ensemble d'entraînement (Train Loss),
- La fonction de perte sur l'ensemble de test (Test Loss).

Ces indicateurs permettent d'évaluer à la fois la qualité de l'apprentissage et la capacité du modèle à généraliser à de nouvelles données.

Comparaison des résultats

Métriques	Regression logistique	Random Forest
Train Accuracy	0.4258	0.9963
Test Accuracy	0.4102	0.4656
Train Loss	1.6857	0.5469
Test Loss	1.7137	1.7200

La régression logistique atteint une précision d'environ 42,6 % sur les données d'entraînement et 41,0 % sur les données de test. Les deux valeurs sont très proches, ce qui indique que le modèle généralise correctement et ne souffre pas de surapprentissage important. La faible différence entre les pertes d'entraînement et de test confirme cette observation. Le modèle présente un comportement stable et cohérent sur les deux ensembles de données. Cependant, les performances globales restent très limitées. Cette limitation est principalement due à la nature linéaire de la régression logistique. Les images du dataset CIFAR-10 contiennent des objets variés dont la reconnaissance nécessite souvent l'identification de formes, de textures et de motifs complexes. Une simple combinaison linéaire des pixels ne permet pas de capturer efficacement ces informations. La régression logistique constitue donc une bonne base de comparaison mais montre rapidement de grosses limites lorsqu'elle est appliquée à un problème de vision par ordinateur.

Les résultats de Random Forest sont très différents. Le modèle obtient une précision quasiment parfaite sur les données d'entraînement avec un score de 99,63 %. Cette performance montre que les arbres de décision sont capables de mémoriser une très grande partie des exemples présents dans le dataset. En revanche, la précision chute à seulement 46,56 % sur les données de test. Bien que ce résultat soit légèrement supérieur à celui de la régression logistique, l'écart extrêmement important entre les performances d'entraînement et de test révèle un phénomène de surapprentissage (overfitting). Le modèle apprend très précisément les exemples d'entraînement mais peine à généraliser sur des images qu'il n'a jamais vues. Cette observation est également visible au niveau de la fonction de perte. La perte d'entraînement est faible tandis que la perte sur les données de test reste élevée. Ce comportement est fréquent avec les modèles basés sur des arbres lorsque la complexité du modèle devient importante. Les arbres mémorisent certaines particularités du jeu d'entraînement au lieu d'apprendre des caractéristiques réellement généralisables.

Cette première étude met en évidence les limites des algorithmes de machine learning classiques pour la classification d'images. La régression logistique offre une bonne capacité de généralisation mais des performances modestes. À l'inverse, Random Forest obtient une meilleure précision mais présente un surapprentissage important. Ces résultats montrent que les approches classiques ont des difficultés à exploiter efficacement l'information contenue dans les images. Bien qu'elles constituent d'excellentes références de comparaison, elles restent limitées pour des tâches de vision par ordinateur complexes telles que la classification du dataset CIFAR-10. L'étude des architectures spécialisées pour le traitement d'images permettra d'explorer des solutions mieux adaptées à ce type de données et d'obtenir des performances plus élevées.

CNN's

Prétraitement des données et protocole expérimental

Avant l'entraînement des différents modèles, les images du dataset sont converties en tenseurs puis normalisées afin de faciliter l'apprentissage et d'accélérer la convergence des réseaux.

Afin d'améliorer la capacité de généralisation des modèles et de limiter le surapprentissage, plusieurs techniques d'augmentation de données sont appliquées. Pour les CNN entraînés depuis zéro, nous utilisons principalement des recadrages aléatoires et des symétries horizontales. Les modèles hybrides pré-entraînés bénéficient quant à eux d'augmentations plus avancées, incluant des rotations aléatoires, des modifications de luminosité et de contraste ainsi que du *Random Erasing*.

Les performances sont principalement évaluées à l'aide de l'accuracy calculée sur le jeu de test. Celui-ci n'est jamais utilisé pour mettre à jour les paramètres du modèle, mais uniquement pour mesurer sa capacité de généralisation au cours et à la fin de l'entraînement. La métrique retenue est l'**accuracy**, correspondant à la proportion d'images correctement classifiées.

Les architectures CNN développées dans le cadre du projet sont entraînées avec l'optimiseur **SGD avec momentum** et un scheduler de type *Cosine Annealing*. Les modèles hybrides pré-entraînés (*MobileNetV2* et *EfficientNet-B0*) utilisent quant à eux l'optimiseur **AdamW**, mieux adapté aux tâches de transfert d'apprentissage.

Les principaux hyperparamètres utilisés sont résumés dans le tableau .

Principaux hyperparamètres utilisés pour chaque architecture.

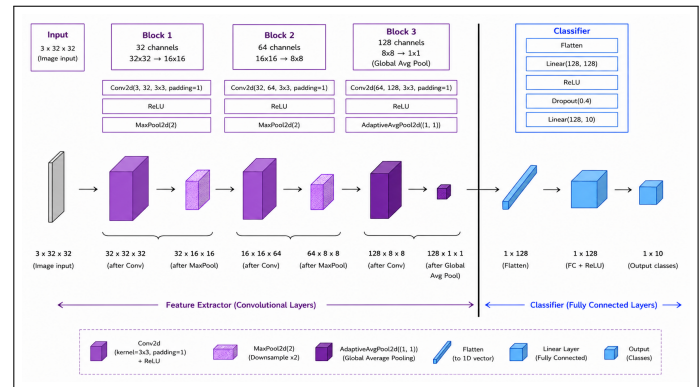
Modèle	Batch Size	Époques	Learning Rate	Optimiseur
CNN référence	128	20	0.1	SGD
CNN référence (50 époques)	128	50	0.1	SGD
DeepCNN	128	20	0.05	SGD
VGGSmall	128	20	0.05	SGD
MiniResNet	128	20	0.05	SGD
MobileNetV2	128	10	10^{-3}	AdamW
MobileNetV2 optimisé	128	15	5×10^{-4}	AdamW
EfficientNet-B0	256	15	5×10^{-4}	AdamW

Architecture 1 : CNN de référence

Le premier modèle étudié est un CNN simple inspiré de celui présenté en TD. Il est composé de couches de convolution, de pooling et de couches entièrement connectées permettant de classer les images parmi les 10 classes de CIFAR-10.

Cette architecture est rapide à entraîner et possède peu de paramètres, mais sa faible profondeur limite sa capacité à capturer des caractéristiques complexes.

Après 20 époques d'entraînement, le modèle atteint une accuracy de **80,2 %**, qui servira de référence pour les expériences suivantes.



Architecture 1.2 : Influence de la couleur

Afin d'évaluer l'importance de l'information colorimétrique, nous conservons la même architecture mais convertissons les images RGB en niveaux de gris.

Le modèle obtient une accuracy de **77,1 %**, soit une baisse de **3,1 points** par rapport aux images couleur. La couleur apporte donc une information discriminante utile à la classification. Nous conserverons ainsi les images RGB pour la suite du projet.

Architecture 1.3 : Augmentation du nombre d'époques

Nous étudions ensuite l'impact de la durée d'entraînement en augmentant le nombre d'époques de 20 à 50, sans modifier l'architecture.

L'accuracy atteint alors **84,4 %**, soit un gain de **4,2 points** par rapport au modèle de référence. Le réseau continue donc d'apprendre au-delà de 20 époques.

Cependant, les gains deviennent progressivement plus faibles et les performances semblent tendre vers un plateau. Cette expérience suggère que les limites du modèle proviennent désormais davantage de son architecture que de son temps d'entraînement.

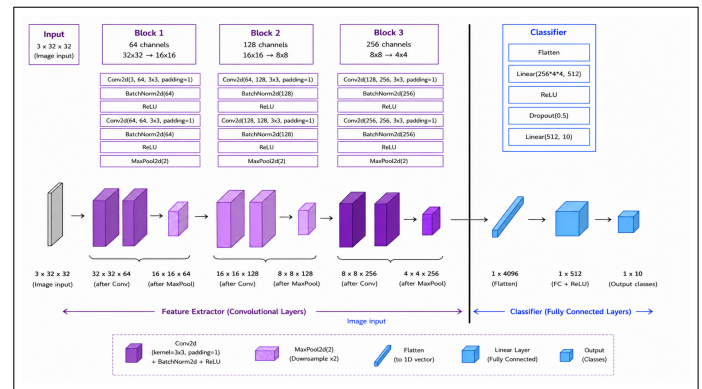
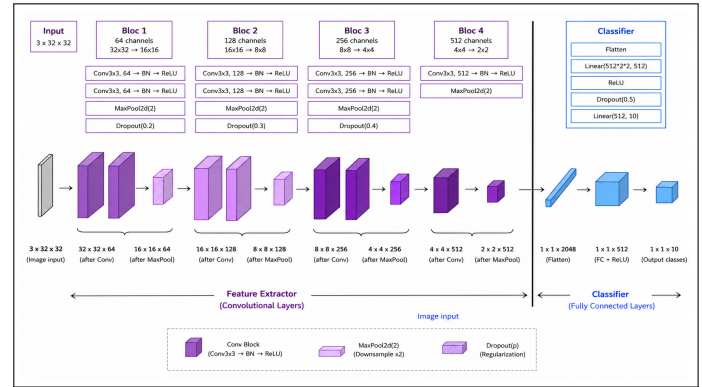
Architecture 2: CNN profond

Nous faisons l'hypothèse que l'architecture initiale n'est pas suffisamment profonde pour capturer l'ensemble des caractéristiques présentes dans les images. Nous concevons donc un CNN plus profond, comportant davantage de couches de convolution et de filtres, afin d'améliorer sa capacité de représentation. L'objectif est de permettre au réseau d'apprendre des caractéristiques plus complexes et plus discriminantes. Le CNN profond obtient une accuracy de **89,1 %** sur le jeu de test, soit un gain de près de **9 points** par rapport au modèle de référence. L'écart entre l'accuracy d'entraînement (89,5 %) et l'accuracy de test (89,1 %) reste très faible, ce qui indique une bonne généralisation et l'absence de surapprentissage significatif.

Ces résultats confirment que les performances du premier modèle étaient principalement limitées par sa profondeur. L'ajout de couches convolutionnelles, combiné à la Batch Normalization et au Dropout, permet d'obtenir une amélioration significative des performances tout en conservant une bonne robustesse.

Architecture 3: VGGSmall

Après avoir étudié un CNN profond conçu manuellement, nous nous intéressons à une architecture inspirée de la famille VGG, reconnue pour ses excellentes performances en classification d'images. Le principe de VGG consiste à empiler plusieurs couches de convolution de taille 3×3 afin d'extraire progressivement des caractéristiques de plus en plus complexes. Cette stratégie permet d'augmenter la profondeur du réseau tout en conservant une architecture simple et homogène. Notre version **VGGSmall** reprend cette philosophie en combinant plusieurs blocs convolutionnels suivis de couches de Max-Pooling, chargées de réduire progressivement la taille des cartes de caractéristiques.



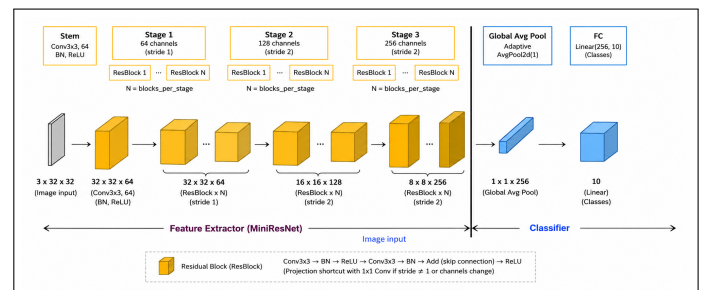
L'objectif de cette expérimentation est de déterminer si une architecture plus profonde et plus structurée permet d'améliorer les performances obtenues avec le DeepCNN, tout en conservant un coût de calcul raisonnable.

Le modèle atteint une accuracy de **90,8 %** sur le jeu de test, soit un gain de près de **2 points** par rapport au DeepCNN (**89,1 %**). Cette amélioration confirme l'intérêt d'une architecture plus profonde pour apprendre des représentations visuelles plus riches et plus discriminantes.

Les résultats montrent également que l'augmentation de la profondeur reste bénéfique tant qu'elle est accompagnée d'une architecture adaptée. VGGSmall constitue ainsi un bon compromis entre simplicité de conception, performances et coût de calcul, tout en préparant la transition vers des architectures plus avancées comme les réseaux résiduels.

Architecture 4 : MiniResNet

L'augmentation de la profondeur d'un réseau améliore généralement ses performances, mais peut rendre l'entraînement plus difficile en raison des problèmes de propagation du gradient. Pour y remédier, nous avons implémenté une version simplifiée de ResNet, appelée *MiniResNet*. Cette architecture introduit des connexions résiduelles (*skip connections*) permettant à l'information de contourner certaines couches et facilitant ainsi l'apprentissage de réseaux plus profonds.



L'objectif est d'évaluer l'apport de ces blocs résiduels par rapport aux architectures CNN précédentes.

Le MiniResNet atteint une accuracy de **92,5 %** sur le jeu de test, soit un gain de plus de **12 points** par rapport au CNN de référence (80,2 %) et de près de **2 points** par rapport à VGGSmall (90,8 %). Les performances progressent régulièrement tout au long de l'entraînement tandis que la loss de test diminue continuellement. C'est le meilleur modèle CNN obtenue dans notre étude.

CNN Hybride

Les architectures précédentes ont été entraînées intégralement sur CIFAR-10. Nous cherchons désormais à exploiter des modèles plus avancés déjà pré-entraînés sur de très grands jeux de données.

Pour cela, nous utilisons le principe du *transfert d'apprentissage*. L'idée consiste à réutiliser des réseaux ayant déjà appris des caractéristiques visuelles générales sur ImageNet, un dataset contenant plusieurs millions d'images. Les premières couches de ces modèles sont capables de détecter efficacement les contours, textures et formes complexes, ce qui permet d'obtenir de bonnes performances même avec un temps d'entraînement réduit.

Deux architectures sont étudiées :

- **MobileNetV2**, un modèle léger conçu pour offrir un excellent compromis entre précision et coût de calcul ;
- **EfficientNet-B0**, une architecture plus récente optimisant simultanément la profondeur, la largeur et la résolution du réseau.

Architecture 5.1 : MobileNetV2Transfer (version initiale)

Cette première version de MobileNetV2 utilise des poids pré-entraînés sur ImageNet. Afin de conserver les connaissances acquises lors du pré-entraînement, la majorité des couches du réseau est gelée et seules les trois dernières couches du bloc de caractéristiques ainsi que le classifieur final sont réentraînés sur CIFAR-10.

Les images sont redimensionnées en 224×224 , conformément au format d'entrée attendu par le modèle original. Cette approche permet de bénéficier des représentations apprises sur un très grand jeu de données tout en limitant le nombre de paramètres à optimiser.

Le modèle atteint une accuracy maximale de **88,7 %** sur le jeu de test. Les performances progressent rapidement grâce au transfert d'apprentissage, mais semblent atteindre un plateau après quelques époques. Bien que ce résultat soit supérieur au CNN de référence, il reste inférieur à ceux obtenus avec le DeepCNN (**89,1 %**), le VGGSmall (**90,8 %**) et le MiniResNet (**92,5 %**).

De plus, le temps d'entraînement est particulièrement élevé (**336 minutes**) pour seulement 10 époques, principalement en raison du redimensionnement des images en 224×224 . Cette première expérimentation montre ainsi que le transfert d'apprentissage ne garantit pas automatiquement de meilleures performances et qu'un réglage plus fin du modèle reste nécessaire.

Architecture 5.2 : MobileNetV2Transfer optimisé

Afin d'améliorer les performances et de réduire le temps d'entraînement, plusieurs modifications ont été apportées à l'architecture précédente.

La taille des images est réduite à 160×160 , ce qui diminue significativement le coût de calcul. Des techniques d'augmentation de données plus avancées sont également introduites (*Random Rotation*, *Color Jitter* et *Random Erasing*) afin d'améliorer la généralisation du modèle. Enfin, le classifieur est remplacé par une couche de sortie précédée d'un Dropout et seules les deux dernières couches du backbone sont réentraînées. Cette version bénéficie également d'optimisations d'exécution telles que le *mixed precision training*, le préchargement des données et l'utilisation d'un scheduler de taux d'apprentissage.

Le modèle atteint une accuracy maximale de **90,3 %** sur le jeu de test, soit un gain de **1,6 point** par rapport à la version initiale. Dans le même temps, le temps d'entraînement est réduit de **336 à 209 minutes** tout en augmentant le nombre d'époques de 10 à 15, ce qui représente une amélioration significative de l'efficacité du modèle.

Ces résultats montrent que l'optimisation des transformations de données et des hyperparamètres a un impact plus important que l'augmentation de la résolution des images. Cette version devient ainsi compétitive avec les architectures CNN développées spécifiquement pour CIFAR-10 et démontre l'intérêt du transfert d'apprentissage lorsqu'il est correctement adapté au problème étudié.

Architecture 6 : EfficientNet-B0

EfficientNet-B0 est une architecture proposée par Tan et Le dans l'article *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* [?]. Contrairement aux approches classiques qui augmentent uniquement la profondeur ou la largeur du réseau, EfficientNet introduit une stratégie de mise à l'échelle équilibrée (*compound scaling*) qui optimise simultanément la profondeur, la largeur et la résolution des images d'entrée.

Cette approche permet d'utiliser plus efficacement les paramètres du modèle et d'obtenir un excellent compromis entre performances et complexité. Comme pour MobileNetV2, nous utilisons une version pré-entraînée sur ImageNet puis adaptée à la classification des 10 classes de CIFAR-10 par transfert d'apprentissage.

Le modèle atteint une accuracy maximale de **92,8 %** sur le jeu de test, ce qui en fait le modèle le plus performant de notre étude. Il dépasse non seulement les architectures CNN classiques comme le DeepCNN (**89,1 %**) et le VGGSmall (**90,8 %**), mais également le MiniResNet (**92,5 %**), qui constituait jusqu'alors la meilleure architecture entraînée uniquement sur CIFAR-10.

Ces résultats montrent que la combinaison d'une architecture moderne et du transfert d'apprentissage permet d'obtenir des performances de très haut niveau sur CIFAR-10. Avec une accuracy supérieure à 92 %, EfficientNet-B0 confirme l'intérêt des modèles pré-entraînés et constitue le modèle le plus performant évalué dans ce projet.

Comparaison des résultats

Cette étude nous a permis de comparer plusieurs approches de classification d'images sur le dataset CIFAR-10, allant des CNN développés manuellement à des architectures modernes pré-entraînées.

L'analyse du dataset a montré que certaines classes possèdent des caractéristiques visuelles complexes et que l'information colorimétrique joue un rôle important dans la classification. L'expérience réalisée sur des images en niveaux de gris a notamment mis en évidence une baisse de performance de 3,1 points par rapport aux images RGB.

Concernant le choix des architectures, nous avons progressivement augmenté la complexité des modèles afin d'évaluer l'impact de leur capacité de représentation. Le CNN de référence obtient une accuracy de 80,2 %, tandis que l'augmentation du nombre d'époques permet d'atteindre 84,4 %. L'utilisation d'architectures plus profondes améliore ensuite significativement les performances : le DeepCNN atteint 89,1 % et le MiniResNet 92,5 %.

Enfin, nous avons étudié l'apport du transfert d'apprentissage à travers MobileNetV2 et EfficientNet-B0. Ces architectures bénéficient de connaissances acquises sur le dataset ImageNet et permettent d'obtenir d'excellents résultats avec un nombre limité d'époques d'entraînement. EfficientNet-B0 obtient notamment la meilleure accuracy de l'étude avec 92,8 %.

Les principaux résultats sont résumés dans le tableau .

Comparaison des performances des différents modèles.

Modèle	Accuracy (%)	Temps d'entraînement
CNN référence (20 époques)	80.2	13min30
CNN référence (50 époques)	84.4	31min30
DeepCNN	89.1	3h22min
VGGsmall	90.8	1h20
MiniResNet	92.5	3h38
MobileNetV2	88.7	5h36
MobileNetV2 (optimisé)	90.3	3h30
EfficientNet-B0	92.8	3h13

Ces résultats montrent que l'augmentation de la profondeur du réseau améliore fortement les performances, mais que les gains les plus importants sont obtenus grâce aux architectures modernes et au transfert d'apprentissage. Parmi tous les modèles étudiés, **EfficientNet-B0** offre les meilleures performances avec une accuracy de **92,8 %**, tandis que **MiniResNet** constitue la meilleure architecture entièrement entraînée sur CIFAR-10.

Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours de ce projet. La principale concernait le temps d'entraînement des modèles les plus complexes, en particulier les architectures pré-entraînées comme MobileNetV2 et EfficientNet-B0. Le redimensionnement des images ainsi que le fine-tuning des modèles ont considérablement augmenté le coût de calcul par rapport aux CNN développés manuellement.

Le choix des hyperparamètres a également représenté un défi important. Les performances obtenues dépendent fortement du taux d'apprentissage, du nombre d'époques, de la taille des batchs ou encore des techniques d'augmentation de données. Plusieurs expérimentations ont été nécessaires afin de trouver des configurations permettant d'obtenir de bonnes performances sans surapprentissage.

Enfin, la comparaison entre les différentes architectures a montré qu'une augmentation de la complexité du modèle ne garantit pas systématiquement une amélioration des résultats. Il a été nécessaire de trouver un compromis entre précision, temps d'entraînement et complexité du réseau. Cette démarche expérimentale a permis de mieux comprendre les avantages et les limites des différentes familles d'architectures convolutives.

Partie II

Présentation des données

La reconnaissance automatique de la parole est un domaine majeur de l'intelligence artificielle visant à convertir un signal audio en information exploitable. Dans ce projet, nous nous intéressons à une tâche simplifiée appelée *keyword spotting*, qui consiste à reconnaître des mots courts prédéfinis.

Le dataset Speech Commands est une base de données audio dédiée à la reconnaissance vocale de mots-clés. Il provient d'un projet de Google, il contient des milliers d'enregistrements de courtes commandes vocales prononcées par différents locuteurs. Chaque échantillon audio est associé à plusieurs informations : le signal audio, la fréquence d'échantillonnage, le label correspondant au mot prononcé, l'identifiant du locuteur et le numéro d'énoncé. Le dataset est utilisé pour entraîner et évaluer des modèles de reconnaissance vocale léger. Dans cette étude nous proposons de nous concentrer sur la reconnaissance du vocabulaire, plutôt que celle du locuteur.

Chargement des données

Listing 1: Chargement du dataset d'entraînement Speech Commands

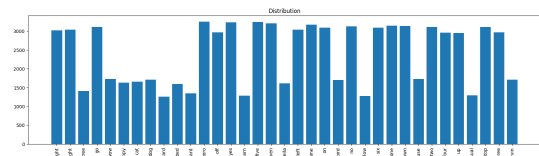
```
1 import torchaudio
2 train = torchaudio.datasets.SPEECHCOMMANDS(download=True, root="./", subset="training")
```

Le dataset fournit par torch est divisé en trois sous-ensemble : Training, Validation, Testing

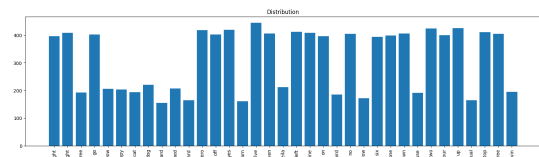
- Nombre d'échantillon d'entraînement : 84843
- Nombre d'échantillon de validation : 9981
- Nombre d'échantillon de test : 11005

Label
right
eight
tree
go
wow
happy
cat
dog
forward
bed
backward
zero
off
yes
learn
five
seven
sheila
left
nine
on
bird
no
follow
six
one
down
house
two
four
up
visual
stop
three
marvin

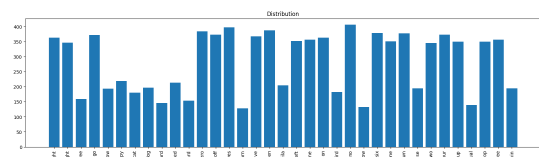
Liste des 35 labels du dataset



Distribution entraînement



Distribution test



Distribution validation

On remarque que les distribution entre les sous-ensembles sont équivalentes.

Visualisation des données

Waveforms

Le signal audio brut peut être représenté sous la forme d'une waveform (forme d'onde), c'est-à-dire une représentation de l'amplitude du signal en fonction du temps. Chaque point du signal correspond à une mesure capturée à un instant précis lors de l'enregistrement. Dans le cas du dataset, les enregistrements sont échantillonnés à une fréquence de 16kHz, ce qui signifie que 16 000 valeurs sont mesurées par seconde. Les waveforms permettent d'observer certaines caractéristiques globales du signal, comme l'intensité de la voix, la durée de la prononciation ou encore les silences présents dans l'audio.

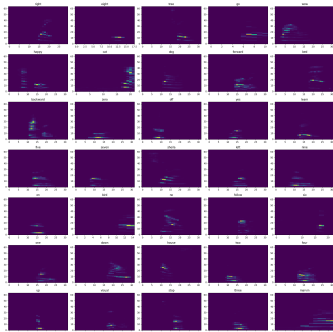
On obtient l'intensité du son à chaque instant, mais cela ne permet pas d'identifier précisément les fréquences présentes dans le signal. Or, dans la parole humaine, les informations essentielles pour différencier les mots sont principalement portées par les variations fréquentielles des sons prononcés. Deux mots peuvent ainsi avoir des waveforms visuellement similaires malgré des contenus fréquentiels très différents. De plus, un même mot peut produire des formes d'onde différentes selon le locuteur, l'intonation ou la vitesse de prononciation. Pour rendre ces caractéristiques plus exploitables par les modèles de classification, il est donc nécessaire de passer à une représentation temps-fréquence.

MelSpectrogrammes

Un spectrogramme est une représentation visuelle des fréquences présentes dans un signal audio au cours du temps. Il est obtenu grâce à une transformation mathématique appelée Short-Time Fourier Transform, qui consiste à découper le signal audio en petites fenêtres temporelles puis à appliquer une transformée de Fourier sur chacune d'elles. Cette opération permet d'observer l'évolution du contenu fréquentiel du signal.

Dans un spectrogramme, l'axe horizontal représente le temps, l'axe vertical les fréquences et l'intensité des couleurs représente l'énergie associée à chaque fréquence. Cette représentation est particulièrement adaptée à la reconnaissance vocale, car elle met en évidence les structures fréquentielles caractéristiques des phonèmes et des mots prononcés.

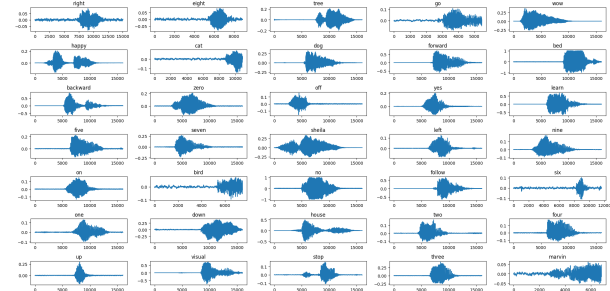
Cependant, l'oreille humaine ne perçoit pas les fréquences de manière linéaire. Les basses fréquences sont perçues avec beaucoup plus de précision que les hautes fréquences. Afin de reproduire ce comportement, nous utilisons des MelSpectrogrammes. Ceux-ci appliquent une échelle de fréquences appelée *échelle de Mel*, qui compresse les hautes fréquences et accorde davantage d'importance aux basses fréquences. Cette représentation est largement utilisée dans les tâches de reconnaissance vocale car elle fournit des caractéristiques plus proches de la perception humaine.



Exemple de MelSpectrogrammes générés à partir des échantillons audio du dataset Speech Commands.

Les Spectrogrammes peuvent être interprétés comme des images. Les CNN peuvent alors détecter des motifs fréquentiels, de la même manière qu'ils détectent des formes ou des textures dans des images classiques.

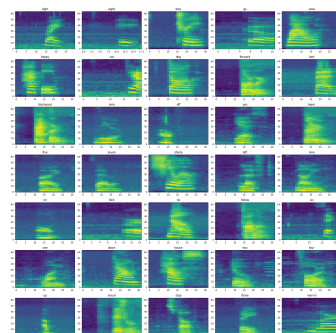
Enfin, nous appliquons une transformation logarithmique sur les spectrogrammes, car certaines fréquences peuvent avoir des amplitudes très élevées tandis que d'autres possèdent des amplitudes beaucoup plus faibles. Le passage au logarithme permet de réduire cet écart et de rendre les détails fréquentiels plus visibles pour le modèle.



Exemple de waveforms provenant du dataset.

Listing 2: Fonction pour le MelSpectrogramme

```
1 self.spec = T.MelSpectrogram(
2   n_fft=1024,           # Taille de la fenetre FFT,
                           # plus cette valeur est grande, meilleure est
                           # la resolution en frequence, mais moins bonne
                           # est la resolution temporelle.
3
4   sample_rate=16000, # Frequence d'echantillonnage
                           # du signal audio (en Hz).
5
6   n_mels=64,           # Nombre de bandes de frequence
                           # de Mel, plus ce nombre est eleve, plus la
                           # representation frequentielle est detaillee.
7
8   hop_length=512       # Nombre d'echantillons entre
                           # deux fenetres FFT consecutives, determine le
                           # chevauchement entre les fenetres et influence
                           # )
```



Exemple de Log-MelSpectrogrammes utilisés comme entrée des modèles de classification.

Nettoyage des données

Ici, nous abordons le problème de la variabilité de la durée des échantillons audio au sein du dataset, que nous avons remarquer lors de notre analyse exploratoire (cf. *ExplorationDonnées.ipynb*). Bien que tous les signaux soient plus courts que 1 seconde, leurs longueurs diffèrent, ce qui peut poser problème pour les modèles nécessitant des entrées de taille fixe. Pour uniformiser les données, nous avons choisi de compléter les signaux avec des zéros (équivalent à du silence) à la fin de chaque échantillon, afin d'atteindre une durée standard de 1 seconde. Cette approche garantit que tous les signaux ont la même longueur, tout en préservant leur contenu. Les échantillons dont la longueur est inférieure à 16 000 sont donc remplis avec des zéros pour atteindre cette taille cible.

Pour optimiser le chargement des données et éviter que le CPU ne devienne un goulot d'étranglement pendant l'entraînement, nous utilisons 8 workers. Ces workers sont des processus parallèles qui chargent et préparent les données en arrière-plan, permettant au GPU de se concentrer sur les calculs du modèle. L'option `persistant_workers=True` maintient ces processus actifs entre les itérations, réduisant ainsi le temps de latence lié à leur initialisation. Enfin, `pin_memory=True` permet de transférer les données directement vers la mémoire du GPU, améliorant encore les performances lors de l'entraînement.

Pipeline d'entraînement

Le pipeline d'entraînement suit une approche structurée pour évaluer et optimiser les modèles. Dans un premier temps, chaque modèle est entraîné sur le jeu de données d'entraînement. À la fin de chaque epoch, le modèle est évalué sur le jeu de validation afin de surveiller sa généralisation et d'éviter le overfitting. Cette étape permet de mettre en place l'early stopping.

Enfin, une fois l'entraînement terminé, les modèles sont testés sur le jeu de test, qui n'a jamais été utilisé pendant les phases d'entraînement ou de validation. Cela garantit une évaluation impartiale et permet de comparer objectivement les performances des différents modèles proposés.

Pour plus de détails sur l'implémentation, se référer au code.

Hyperparameter Tuning

Dans cette section, nous détaillons la méthode utilisée pour optimiser les hyperparamètres des modèles présentés. *Nous ne nous concentrons pas sur l'optimisation des architectures, car en IA01 nous avons déjà proposer une approche pour la recherche d'architectures de réseaux de neurones.*

Pour plus de détails sur l'implémentation, se référer au code.

Processus d'optimisation

Chaque modèle a vu ses hyperparamètres être fine-tuné avec **Optuna** selon les critères suivants :

- **Workers** : 8 pour l'entraînement et 8 pour la validation, afin de paralléliser efficacement les calculs.
- **Matériel** : L'ensemble du traitement est réalisé sur **GPU** pour accélérer les itérations.
- **Paramètres optimisés** : `BatchSize`, `LearningRate` et `WeightDecay` sont recherchés dynamiquement.
- **Nombre d'essais** : 50 tentatives avec Optuna, chaque essai exécutant 3 *epochs* pour évaluer rapidement la performance.

Nous avons utiliser aussi la capacité d'Optuna a utiliser le *pruning*, une technique qui arrête prématurément les essais peu prometteurs, économisant ainsi du temps et des ressources.

Méthodes statiques des modèles

Chaque modèle implémente deux méthodes statiques :

- `optimize` : Génère les paramètres pour un essai Optuna.
- `optimize_defaults` : Attribue des paramètres par défaut au modèle si Optuna n'est pas utilisé. Ces paramètres peuvent être mis à jour ultérieurement pour refléter les meilleurs résultats trouvés par Optuna.

Pour une compréhension approfondie de cette méthode, nous vous invitons à consulter les implémentations des modèles en annexe.

Nous sommes conscients que cette approche ne couvre pas toutes les possibilités d'optimisation. Cependant, elle établit une base solide et démontre notre capacité à concevoir des modèles compatibles avec des outils comme Optuna.

Modèles

CNN

Motivation

Les CNN sont utilisés en traitement du signal audio, notamment pour des tâches comme le Keyword Spotting. Leur capacité à extraire des caractéristiques locales à partir de représentations comme les spectrogrammes, en fait un choix évident pour cette application. En appliquant des filtres convolutifs sur les spectrogrammes, les CNN peuvent identifier des motifs fréquentiels spécifiques aux mots-clés, tout en étant robustes aux variations de prononciation ou de bruit. De plus les CNN offrent un bon compromis entre précision et efficacité, ce qui les rend adaptés à des tâches en temps réel comme la détection de mots-clés.

Définition

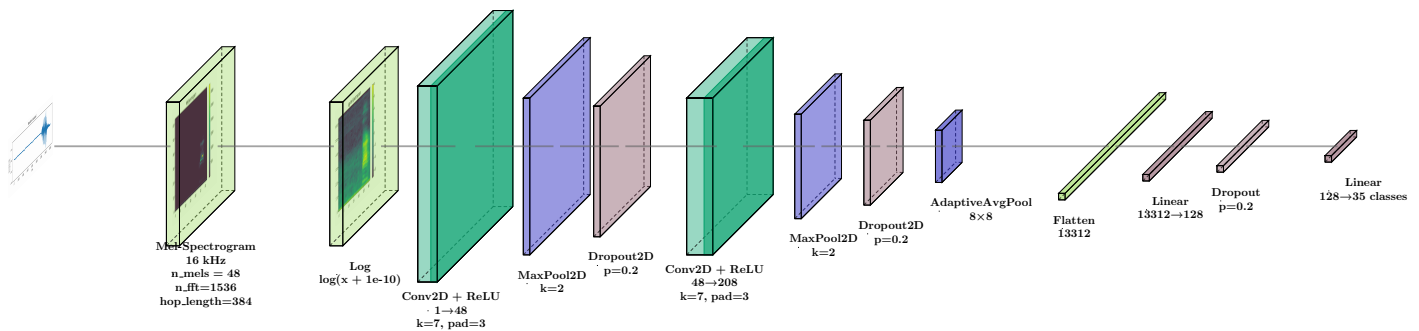
Au départ, nous avons testé une architecture conçue de manière empirique, qui a déjà donné de très bons résultats. *Nous nous sommes rendu compte, après le fine-tuning, qu'il manquait un paramètre optimisable : la taille de la couche intermédiaire (128).*

Résultats optima pour le CNN classique :

Listing 3: Résultat optuna

```
1 Meilleur parametres : {'batch_size': 16, 'lr': 0.00010012603121343938, 'weight_decay':
  1.6319322258025493e-07, 'n_mels': 48, 'n_fft': 1536, 'hop_length': 384, 'cnn1_out_channels': 48,
  'cnn2_out_channels': 208, 'kernel_size': 7, 'pooling_kernel': 2, 'adaptive_pool_output': (8, 8)
  , 'dropout': 0.2}
2 Meilleur accuracy : 0.8939985973349364
```

On se retrouve donc avec la forme suivante :



Architecture pour le CNN

Pour voir l'implémentation se référer au code.

Pour les modèles CNNTrad et CNNOne, inspirés d'un article de Google, les résultats n'ont pas été à la hauteur de nos attentes, car ces architectures n'étaient pas adaptées à nos données. Néanmoins, nous avons relevé le défi de les implémenter pour explorer leur potentiel. **Papier relatif**

Nous proposons donc d'implémenter les deux architectures proposer dans le papier.

CNNTrad

Listing 4: Exemple d'utilisation de la fonction

```
1 Meilleur parametres : {'batch_size': 16, 'lr': 0.00021495551281313965, 'weight_decay':
  7.109975589372269e-07, 'dropout': 0.0}
2 Meilleur accuracy : 0.8765654744013626
```

Pour voir l'implémentation se référer au code et au papier relatif

CNNOne

Listing 5: Exemple d'utilisation de la fonction

```
1 Meilleur parametres : {'batch_size': 32, 'lr': 0.0006425519426416211, 'weight_decay':
  2.929655627975144e-05, 'dropout': 0.0}
2 Meilleur accuracy : 0.7901011922653041
```

Pour voir l'implémentation se référer au code et au papier relatif

CNN BiLSTM

Motivation

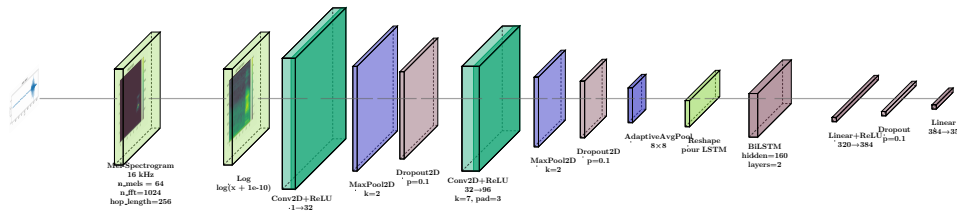
Les CNN sont efficaces pour extraire des caractéristiques locales à partir de spectrogrammes, mais ils ne capturent pas les dépendances temporelles à long terme dans les signaux audio. Les réseaux récurrents, comme les BiLSTM (Bidirectional Long Short-Term Memory), sont conçus pour modéliser ces dépendances en traitant les séquences dans les deux sens (avant et arrière). En combinant les CNN pour l'extraction de caractéristiques spectrales et les BiLSTM pour la modélisation temporelle, nous proposons une architecture hybride qui pourrait améliorer la reconnaissance des mots-clés. Cette approche nous semble prometteuse, car elle permet de tirer parti des forces des deux types d'algorithme : la capacité des CNN à détecter des motifs locaux dans les spectrogrammes et celle des BiLSTM à capturer le contexte temporel des séquences audio.

Définition

Listing 6: Résultat optuna

```
1 Meilleur parametres : {'batch_size': 64, 'lr': 0.0010061137930597018, 'weight_decay':
  1.1559805487396425e-06, 'n_mels': 64, 'n_fft': 1024, 'hop_length': 256, 'cnn1_out_channels': 32,
  'cnn2_out_channels': 96, 'kernel_size': 7, 'pooling_kernel': 2, 'adaptive_pool_output': (8, 8),
  'lstm_hidden_size': 160, 'lstm_num_layers': 3, 'fc_hidden_size': 384, 'dropout': 0.1}
2 Meilleur accuracy : 0.9217513275222924
```

On se retrouve avec l'architecture suivante, *pour voir l'implémentation se référer au code*.



Architecture pour le CNN BiLSTM

CNN 2D & 1D

Motivation

Les CNN 1D peuvent être appliqués directement sur les waveforms (signaux audio bruts) pour extraire des caractéristiques temporelles. En combinant celui-ci avec le CNN 2D, nous cherchons à exploiter à la fois les informations temporelles (1D) et spectrales (2D) pour améliorer la robustesse et la précision du modèle. Cette hybridation devrait permettre de tirer parti des forces de chaque méthode : la capacité des CNN 1D à modéliser les dépendances temporelles locales et celle des CNN 2D à analyser les motifs fréquentiels.

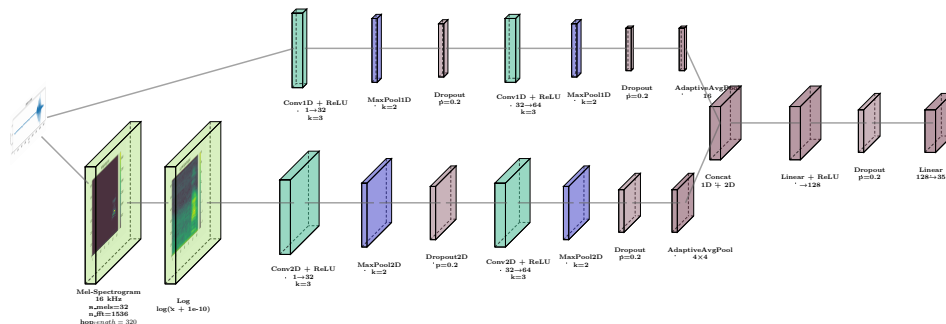
On a aussi vu quelques projet qui utiliser ce genre d'astuce, on a souhaiter voir si cela était pertinent dans notre cas.

Définition

Listing 7: Résultat optuna

```
1 Meilleur parametres : {'batch_size': 32, 'lr': 0.00012765384089860127, 'weight_decay':
  0.00011068942908456975, 'n_mels': 32, 'n_fft': 1536, 'hop_length': 320, 'cnn1d_out_channels_1':
  32, 'cnn1d_out_channels_2': 64, 'kernel_size_1d': 7, 'pooling_kernel_1d': 4, '
  adaptive_pool_1d_output': 12, 'cnn2d_out_channels_1': 128, 'cnn2d_out_channels_2': 224, '
  kernel_size_2d': 7, 'pooling_kernel_2d': 3, 'adaptive_pool_2d_output': (8, 8), 'fc_hidden_size':
  320, 'dropout': 0.30000000000000004}
2 Meilleur accuracy : 0.8997094479511071
```

On se retrouve avec l'architecture suivante, *pour voir l'implémentation se référer au code*.



Architecture pour le CNN1D2D

Comparaisons des résultats

Évaluons les performances des différents modèles. Dans un premier temps, chaque architecture a été générée et entraînée sur 20 époques. Les métriques retenues pour l'évaluation sont l'accuracy, le F1-Score et la loss, calculées sur le jeu de données de test. Par ailleurs, nous avons optimisé les hyperparamètres de chaque modèle à l'aide de Optuna, en fixant un learning rate et un weight decay spécifiques pour chaque architecture.

Les temps d'optimisation (Optuna), d'entraînement et de test ont également été mesurés afin d'évaluer l'efficacité de chaque approche. Les résultats obtenus sont présentés dans les tableaux ci-dessous. Le premier tableau à gauche compare les performances des modèles en termes de métriques classiques, tandis que le tableau de droite détaille les hyperparamètres utilisés pour l'optimiseur Adam. Enfin, le dernier tableau résume les temps de calcul associés à chaque étape.

Comparaison des performances sur Speech Commands.

Modèle	Accuracy (%)	F1-Score	Loss
CNN	92.28	0.9227	0.2795
BiLSTM	92.86	0.9289	0.2355
CNN1D2D	85.03	0.8497	0.4940
CNNTrad	87.10	0.8714	0.4562
CNNOne	83.56	0.8360	0.6387

Comparaison des temps sur Speech Commands.

Modèle	Temps optuna	Temps train (min)	Nombre époques	Temps test (s)
CNN	1h13	8.6	19	4.55
BiLSTM	1h23	3.71	11	5.06
CNN1D2D	1h59	9.23	20	5.07
CNNTrad	1h22	5.21	11	4.66
CNNOne	1h05	5.25	20	4.63

Paramètres de l'entraînement

Modèle	Learning Rate	Weight Decay	Batch Size
CNN	0.00010012603121343938	1.6319322258025493e-07	16
BiLSTM	0.0010061137930597018	1.1559805487396425e-06	64
CNN1D2D	0.00012765384089860127	0.00011068942908456975	32
CNNTrad	0.00021495551281313965	7.109975589372269e-07	16
CNNOne	0.0006425519426416211	2.929655627975144e-05	32

Les résultats obtenus sur le jeu de données montrent que le BiLSTM et le CNN classique se valent en termes de performance, avec des accuracies respectives de 92,86% et 92,28 %, ainsi que des losses très proches (0,2355 et 0,2795). Cependant, le BiLSTM se distingue par une convergence plus rapide, avec un temps d'entraînement de 3,71 minutes contre 8,6 minutes pour le CNN, après des optimisations par Optuna de 1h23 et 1h13 respectivement. À l'inverse, le CNN1D2D, moins performant (85,03 % d'accuracy), est le plus coûteux en ressources, avec un temps d'optimisation de 1h59 et un temps d'entraînement de 9,23 minutes. Le CNNTrad et le CNNOne offrent des compromis : le premier allie une accuracy correcte (87,10 %) à un temps d'entraînement modéré (5,21 minutes), tandis que le second, bien que moins précis (83,56 %), se distingue par son efficacité, avec un temps de test de 4,63 secondes et une optimisation de 1h05.

Ces résultats soulignent bien le dilemme entre performance et efficacité, ici le BiLSTM et le CNN représentant les meilleurs équilibres.

Pour voir l'implémentation se référer au code.

Augmentation des données

Pour améliorer la robustesse et la généralisation de nos modèles, nous avons mis en place une augmentation des données directement dans le pipeline de chargement du dataset Speech Commands. Cette approche consiste à appliquer des transformations aléatoires aux échantillons audio pendant l'entraînement, afin de simuler des variations naturelles comme le bruit, les décalages temporels ou les variations de volume, et ainsi enrichir artificiellement la diversité des données.

Notre classe `AugmentedSpeechCommands`, qui hérite de `Dataset`, applique pour chaque échantillon une augmentation choisie aléatoirement parmi plusieurs options. L'ajout de bruit blanc introduit un bruit gaussien de faible amplitude pour simuler des environnements bruyants. Le décalage temporel décale le signal de plus ou moins 10% de sa longueur, avec un remplissage par des zéros pour conserver la taille d'origine, ce qui permet de rendre le modèle invariant aux petits décalages dans la prononciation. La modification du gain ajuste le volume du signal en le multipliant par un facteur aléatoire entre 0,8 et 1,2, puis en le clampant pour éviter les distorsions.

Ces augmentations sont appliquées à la volée pendant l'entraînement, sans modifier le dataset original, ce qui génère des variantes uniques à chaque epoch. Cette méthode a permis d'améliorer les performances, comme en témoignent les résultats obtenus après optimisation avec Optuna.

Toutes les logs sont disponibles sur le [GitHub](#) et le code de l'augmentation

Résultats / Impact

Comparaison des performances sur Speech Commands.

Modèle	Accuracy (%)	F1-Score	Loss
CNN	92.09	0.9207	0.2654
BiLSTM	94.24	0.9424	0.2032
CNN1D2D	90.69	0.9067	0.3143
CNNTrad	89.73	0.8972	0.3882
CNNOne	82.92	0.8292	0.6090

Comparaison des temps sur Speech Commands.

Modèle	Temps optuna	Temps train (min)	Nombre époques	Temps test (s)
CNN	57min	8.55	20	4.41
BiLSTM	1h08	6	19	5.13
CNN1D2D	1h42	17.45	18	5.89
CNNTrad	1h06	8.56	20	4.42
CNNOne	52min	3.8	20	4.67

Paramètres de l'entraînement

Modèle	Learning Rate	Weight Decay	Batch Size
CNN	0.00021318566351811254	1.8942967474553665e-07	16
BiLSTM	0.0002509266876598951	2.34498174330265e-05	64
CNN1D2D	7.742854225812231e-05	9.290133717150751e-06	16
CNNTrad	0.00016171109744381155	4.296880335700137e-05	16
CNNOne	0.000714525852823576	4.807148281042069e-07	64

Les résultats obtenus après l'application de l'augmentation des données confirment son efficacité pour améliorer la généralisation des modèles. Le BiLSTM et le CNN classique restent les architectures les plus performantes, avec des accuracies respectives de 94,24 % et 92,09 % sur le jeu de test, contre 92,86 % et 92,28 % sans augmentation. Le gain est particulièrement marqué pour le BiLSTM, qui bénéficie davantage de la diversité accrue des données pour capturer les dépendances temporelles.

Cependant, l'augmentation des données a un coût computationnel, car les temps d'optimisation avec Optuna et d'entraînement ont légèrement augmenté, notamment pour les architectures hybrides comme le CNN1D2D. Malgré cela, le compromis reste favorable, car l'amélioration des métriques comme le F1-Score et la loss justifie l'investissement en ressources.

Nous avons testé plus d'epochs pour Optuna mais cela n'a pas montré de gains significatifs (92.09% vs 92.38% pour le CNN avec 3 epochs et 8 epochs), confirmant que 3 epochs suffisent pour évaluer rapidement la performance des hyperparamètres.

Nouvelles données

Nos échantillons

Nous proposons la possibilité de tester le modèle sur un enregistrement en direct. (cf . `direct_test.py`)

Conclusion

Cette étude a mis en lumière plusieurs défis majeurs, mais aussi des acquis précieux pour la suite. L'un des obstacles les plus marquants a été l'utilisation initiale du CPU pour l'entraînement des modèles et l'optimisation via Optuna, entraînant des temps de calcul très longs, jusqu'à plus de 7 heures par modèle, avec des plantages fréquents. Le passage au GPU a radicalement amélioré l'efficacité, réduisant les temps d'optimisation et d'entraînement à des durées plus raisonnables, de l'ordre de 1h à 1h30, soulignant ainsi l'importance cruciale de l'accélération matérielle pour ce type de tâches.

Un autre problème a été l'oubli de certains hyperparamètres lors du fine-tuning, comme la taille de la couche intermédiaire dans le modèle CNN, ou encore l'absence d'optimisation des fonctions d'activation et d'autres paramètres architecturaux. Ces lacunes ont révélé une force majeure de notre approche : sa modularité. En effet, nos pipelines, qu'il s'agisse d'Optuna, des tests ou de l'évaluation, ont été conçus pour s'adapter dynamiquement aux paramètres des modèles, sans nécessiter de modifications structurelles. Cette flexibilité ouvre la voie à des améliorations futures, comme l'automatisation complète du processus, de l'optimisation à l'évaluation, voire la modification automatique des paramètres par défaut dans les classes des modèles après optimisation. On pourrait également envisager d'étendre cette modularité à l'optimisation de l'architecture elle-même, comme exploré dans le projet d'IA01.

Par ailleurs, nous avons testé des architectures inspirées des travaux de Google pour le keyword spotting, conçues pour réduire l'empreinte écologique. Cependant, leur utilisation telle quelle, avec les hyperparamètres proposés dans leur papier, n'a pas donné les résultats escomptés. En effet, bien que ces architectures soient prometteuses, elles nécessitent un fine-tuning adapté à nos données pour être pleinement efficaces. Les performances obtenues (88 % d'accuracy) restent acceptables, mais en production, on cherche toujours la meilleure métrique et le temps de calcul le plus rapide. Sachant qu'à chaque application, une métrique différente pourrait être nécessaire : on pourrait accepter une tolérance faible pour des petites tâches, comme reconnaître le prénom d'un assistant vocal, mais exiger une précision bien plus élevée pour des actions critiques, comme le contrôle d'un engin militaire.

De plus, l'augmentation des données s'est révélée être un élément pertinent pour améliorer la généralisation des modèles. En simulant des variations réalistes comme le bruit, les décalages temporels ou les variations de volume, nous avons pu enrichir artificiellement le dataset et réduire le risque de surapprentissage. Cette approche a non seulement boosté les performances (notamment pour le BiLSTM, passant de 92,86 % à 94,24 % d'accuracy), mais a aussi démontré que des transformations simples, appliquées à la volée, suffisent à rendre les modèles plus robustes face aux imperfections des données réelles.

Ce projet a aussi renforcé notre rigueur méthodologique : une syntaxe claire, une vérification systématique des données et une approche itérative pour tester différentes architectures. Ces pratiques ont permis de construire des pipelines réutilisables et modulaires, où 80 % du travail réside dans la préparation, la structure, l'automatisation et l'analyse des données, et seulement 20 % dans la création des modèles, leur optimisation et l'interprétation des résultats.

Enfin, des questions subsistent sur la représentativité du dataset, notamment en ce qui concerne la diversité des accents et des timbres de voix, ainsi que son adéquation pour des cas d'usage réels. Dans une perspective de continuité du projet, il serait intéressant d'explorer plusieurs pistes pour affiner encore les performances, tester d'autres modifications d'augmentation, évaluer l'impact d'un jeu de test fixe par rapport à des altérations aléatoires pour vérifier si le caractère aléatoire de ces transformations est réellement bénéfique, ou encore expérimenter le découpage de signaux audio continus. Une optimisation poussée des architectures pourrait également être envisagée pour mieux répondre aux spécificités des données.

Bibliographie

- <https://alexlenail.me/NN-SVG/LeNet.html>
- <https://www.overleaf.com/project/5ee514f1e88684000165e921>
- Github du projet
- https://www.isca-archive.org/interspeech_2015/sainath15b_interspeech.pdf